Research paper:

Peter Axelsson (2000), "DEM Generation from Laser Scanner Data Using Adaptive TIN Models", ISPRS Archives, Volume XXXIII, Part B4.

The paper introduced a ground-classification method that starts with a few reliable lowest ground points, builds a coarse terrain TIN underneath the point cloud and progressively adds points that fit the expected ground surface.
A version of this algorithm is implemented in TerraScan.



Adaptive_Ground_detection_post_FLAI

Method summary:

1. The tool starts with widely spaced points that are likely to be ground.
2. It connects them into a basic terrain surface and gradually adds new points that fit the expected terrain angle and distance.
3. It then recovers supported hills and ground beneath vegetation, removes isolated spikes, and selects the original points that produce the smoothest terrain model.
4. Finally, it adds more points close to the model so the ground is dense but remains only one surface thick.
5. Everything above that surface is classified by its height above ground.

3 main processing phases:

Terrain discovery	Where does the natural ground surface exist?
Surface cleaning	Which points provide the smoothest reliable TIN?
Dense classification	Which additional returns genuinely belong to that thin ground surface?

A. Prepare the data

1. Find the LAS and LAZ files

Core functions:

```
None
discover_inputs()
read_input_metadata()
```

The program looks for every `.las` and `.laz` file that needs processing.

It reads basic information from each file:

- Number of points.
- Minimum and maximum coordinates.

- Point format.
- Approximate tile position.
- Whether neighbouring files are available.

At this point, it reads only the file header. It does not load millions of points unnecessarily.

Why this is needed

Before processing begins, the tool needs to understand:

- How many files exist.
- Which files are next to each other.
- How much memory may be required.
- How many files can safely run simultaneously.

---

2. Add 10 m of neighbouring-tile context

Core functions:

```
None
find_neighbour_sources()
build_halo_caches()
load_halo_points()
```

A target tile does not know what happens immediately outside its boundary.

The script temporarily loads a 10 m strip of points from neighbouring tiles:

```
None
Neighbour tile → 10 m context → TARGET TILE → 10 m context →
Neighbour tile
```

These neighbouring points help build triangles across the tile boundary.

They influence classification, but they are not written into the target output.

Why this is needed

Without neighbouring context:

- Ground triangles can stop at the tile boundary.
- The edge of one tile may not match the next tile.
- Ground may be missing or spiky along seams.

With 10 m context, the algorithm can see enough terrain on both sides of the boundary to make a better decision.

---

3. Divide large files into manageable work areas

Core functions:

```
None
processing_windows()
buffered_indices_for_window()
process_file_windows()
```

If a file is too large to process comfortably, it is divided into 500 m core areas.

Each core receives an additional 80 m working overlap.

```
None
500 m core A → 80 m shared working area → 500 m core B
```

The algorithm classifies using both the core and overlap points, but writes only the non-overlapping core.

Why 80 m?

The initial seed grid is 60 m.

An 80 m overlap is larger than the seed grid, so every core has enough surrounding information to:

- Select sensible seeds.
- Build complete triangles.
- Fit local terrain planes.
- Avoid internal processing seams.

Difference between the two context values

Setting	Purpose
10 m neighbouring context	Connects separate LAS/LAZ files
80 m internal overlap	Connects processing windows inside one large file

They solve similar boundary problems at different levels.

---

4. Reset the previous ground and vegetation classes

Core function:

```
None
reset_input_class_mask()
```

Points currently in these classes are returned to class 0:

- 2 Ground
- 3 Low vegetation
- 4 Medium vegetation
- 5 High vegetation
- 13 Medium vegetation 2

Other existing classes are preserved, including:

- Class 7 noise.
- Class 14 man-made.
- Class 15 network.
- Other project-specific protected classes.

Why this is needed

If existing class 2 points were automatically treated as correct ground, previous classification mistakes would control the new TIN.

Resetting these five classes gives the algorithm a clean starting point.

---

5. Create a simplified lower representation of the cloud

Core functions:

```
None
lowest_per_grid()
remove_unsupported_low_work_points()
```

In simple terms

The script divides the data into 1 m cells and temporarily selects the lowest candidate in every cell.

```
None
Many original returns
        ↓
One local low point per 1 m cell
        ↓
Structural work set
```

This is not the final ground classification.

It is a simplified set of points used to discover the approximate terrain.

Why not use every point?

Using every point in every TIN iteration would:

- Be significantly slower.
- Use much more memory.
- Allow low vegetation to influence the model.
- Make the TIN chase very small elevation variations.

The 1 m work set gives the terrain algorithm a cleaner and more manageable lower representation.

Conservative isolated-low check

The function `remove_unsupported_low_work_points()` checks for an extremely low point that:

- Is at least 1 m below a reliable local plane.
- Is isolated.
- Has no nearby points supporting that elevation.

It does not immediately classify the point as noise. It temporarily removes it from the initial TIN and checks it again after the ground surface is complete.

B. Discover the terrain

6. Select the initial ground seeds

Core function:

```
None
lowest_per_grid(..., spacing=60.0)
```

The script places a 60 m grid over the work set and chooses the lowest point in each cell.

These points become the first assumed ground points.

Why 60 m?

The value should be similar to the size of the largest object that the TIN must pass underneath.

For example, if a building is 40 m wide, a 60 m area is likely to contain at least one point outside that building that reaches real terrain.

This corresponds to TerraScan's maximum building size control.

What happens if there are too few seeds?

If fewer than four seeds are available, the script makes the grid smaller until enough starting points are found.

---

7. Grow the main progressive TIN

Core functions:

```
None
run_ptd_pass()
build_tin()
accepted_candidates()
triangle_metrics()
adaptive_angle_limit()
```

The script connects the initial seeds into triangles.

It then checks nearby points and asks:

> Does this point look like a sensible continuation of the current terrain?

A point must pass several tests before it can be added.

Pass 1 settings

Test	Value
Maximum distance to triangle	1.40 m
Maximum iteration angle	6°
Maximum terrain angle	88°
Reduce-angle triangle edge	5 m
Allowed distance below TIN	0.15 m

Distance test

The script calculates how far the candidate is from the triangle plane.

If the point is too far from the triangle, it is not added.

Angle test

The script also measures how abruptly the surface would need to change to reach that point.

A small angle represents a gradual terrain change. A large angle may indicate:

- Vegetation.
- A building.
- An isolated object.
- A false upward jump.

One point per triangle

During Pass 1, only the best candidate in each triangle is accepted during one iteration.

The TIN is then rebuilt before more points are considered.

This prevents a large triangle from accepting an entire vegetation layer in one step.

---

8. Run a stricter second pass

Core function:

```
None
run_ptd_pass()
```

The same function is used again, but with stricter settings:

Test	Pass 1	Pass 2
Iteration angle	6°	4°
Distance	1.40 m	0.40 m
Reduce-angle edge	5 m	3 m

In simple terms

Pass 1 finds the general terrain shape.

Pass 2 examines the remaining gaps using a more complete TIN and much tighter acceptance limits.

It is similar to saying:

> We now understand the general hill. Let us carefully check which remaining points genuinely belong to its surface.

Why it helps

Pass 2 can fill real ground:

- Under trees.
- Between sparse LiDAR returns.
- Along continuous slopes.
- Around small undulations.

Because its limits are stricter, it is less likely to accept vegetation.

---

9. Recover ground that the initial TIN passed underneath

Core functions:

```
None
recover_supported_upper_surface()
recover_local_lower_surface()
interpolate_hag_at_indices()
local_plane_support()
```

A normal progressive TIN can still miss genuine terrain.

This commonly happens on a broad convex hill.

The hilltop can sit well above the coarse triangle, causing genuine ground to fail the original distance test.

9A. Lower-envelope recovery

If a local work point is between 0.15 and 2 m below the preliminary TIN, it may be added to correct a triangle that has passed over a depression.

9B. Anchored smooth-surface recovery

The function:

```
None
recover_supported_upper_surface()
```

looks for points slightly above the current model.

A point is considered only when:

- It is close to an existing ground-model point.
- Nearby points form a sensible local surface.
- Several nearby points support the same surface.

This allows the model to climb legitimate terrain without jumping directly into a tree.

9C. Hill and slope recovery

The function:

```
None
recover_local_lower_surface()
```

fits a local plane through nearby lower work points.

If a candidate agrees with that local surface—or has several nearby points with the same behaviour—it can be added.

Why local support matters

One isolated point may be vegetation.

A group of neighbouring points forming a continuous surface is much more likely to represent terrain.

---

10. Remove false canopy points and upward spikes

Core functions:

```
None
prune_model_above_lower_plane()
prune_upward_model_spikes()
```

In simple terms

Terrain recovery deliberately allows the TIN to grow. After that growth, the script checks whether it accidentally climbed into vegetation.

Two cleaning tests are used.

Lower-plane check

The script asks:

> Is this accepted model point sitting above a cleaner, lower terrain surface nearby?

If yes, it may be a canopy or bush point and can be removed.

Spike check

The script compares a model point against:

- Nearby model points.
- Their median elevation.
- A fitted local plane.
- Other lower work points around it.

The point is removed only if:

- It rises above the local median.
- It rises above the local plane.
- There are not enough nearby points supporting that rise.

Why genuine ridges can remain

A real ridge normally has multiple neighbouring points at similar elevations.

An isolated vegetation return normally does not.

The support test is intended to distinguish those two cases.

C. Build and classify the final surface

11. Choose the smoothest representative ground points

Core function:

```
None
select_smooth_model_representatives()
```

The progressive TIN has now discovered where the terrain exists, but its model points may still contain small measurement scatter.

For every terrain cell, the script chooses the original LiDAR point that best fits the locally estimated terrain plane.

Adaptive spacing

The representative density changes with terrain slope:

Terrain	Approximate spacing
Flat terrain	0.80 m
Sloping terrain	0.60 m

Slopes receive more representative points because they require more detail to describe their shape correctly.

---

12. Construct the final clean ground TIN

Core function:

```
None
build_tin()
```

The selected representatives are connected into the final clean terrain surface.

This TIN is used for:

- Ground membership decisions.
- HAG calculation.
- Vegetation classification.
- Final DEM behaviour.

The representatives control the shape of the terrain, but they are not necessarily all the final class 2 points.

---

13. Add more ground points without thickening the surface

Core functions:

```
None
select_adaptive_ground_membership()
interpolate_tin_membership_context()
build_planar_tin()
```

The representative TIN is smooth, but it may contain fewer class 2 points than required.

The script therefore creates a very thin acceptance band around the clean TIN.

Only points inside this narrow band are considered for additional ground membership.

Approximate envelope thickness

Terrain	Below TIN	Above TIN
Flat	0.06 m	0.04 m
Steep	0.10 m	0.08 m

The distance is measured perpendicular to the triangle surface. This is more suitable than vertical distance on steep slopes.

Progressive density

The script still does not accept every point in the band immediately.

It adds the best point in each triangle, rebuilds the density TIN and repeats until the triangles are sufficiently small.

Target triangle edges are approximately:

Terrain	Target edge
Flat	0.96 m
Steep	0.63 m

This produces:

- More ground points on slopes.
- A useful but controlled density.
- A one-surface-thick class 2 layer.
- Less chance of low vegetation becoming ground.

This stage is what allows the result to be both smooth and reasonably dense.

---

14. Calculate HAG and vegetation classes

Core functions:

```
None
interpolate_hag()
assign_hag_classes()
```

For every point, the script finds the final ground elevation beneath it.

The low-vegetation blanket

Because the class 2 envelope is only a few centimetres thick, most points immediately above the ground remain class 3.

---

15. Write output points

Core functions:

```
None
apply_window_result()
ensure_hag_dimension()
las.write()
```

Languages and libraries

Component	Language	Main technology
Optional launcher	Windows Batch	Calls Python and reports success/failure
Algorithm and scheduling	Python 3	Functions, classes and multiprocessing
LAS/LAZ reading/writing	Python	`laspy`
Array processing	Python API backed by compiled code	NumPy
TIN construction	Python API backed by Qhull	`scipy.spatial.Delaunay`
Neighbour searching	Python API backed by compiled KD-tree	`scipy.spatial.cKDTree`
Parallel files	Python	`ProcessPoolExecutor`
Parallel internal windows	Python	`ThreadPoolExecutor`
LAZ compression, when required	Python interface to compiled codec	`lazrs`
Temporary neighbour cache	Python/NumPy	`np.memmap`

For LAS input, `lazrs` is not used.

Step-by-step implementation

1. Read the point cloud

Language: Python 3 with `laspy`.

Core commands:

```
None
las = laspy.read(input_path)

x = np.asarray(las.x, dtype=np.float64)
y = np.asarray(las.y, dtype=np.float64)
z = np.asarray(las.z, dtype=np.float64)
classification = np.asarray(las.classification, dtype=np.uint8)
```

`laspy.read()` reads the LAS/LAZ header, point coordinates, classification and other dimensions.

`np.asarray()` exposes those dimensions as efficient numeric arrays. The original XYZ coordinates are never modified.

For large neighbour files, the script uses:

```
None
with laspy.open(source_path) as reader:
    for points in reader.chunk_iterator(...):
```

This streams the file in chunks instead of loading another complete neighbour tile into RAM.

---

2. Discover neighbouring tiles

Language: Python with SciPy.

Core commands:

```
None
tree = cKDTree(tile_centres)
positions = tree.query_ball_point(target_centre, search_radius)
```

A KD-tree is a spatial index. It quickly finds candidate files located near the target tile.

The script then performs an exact bounding-box intersection check, so the KD-tree search only produces candidates; it does not decide the final adjacency by itself.

Numeric filenames such as:

```
None
468500_4966000.las
```

are interpreted as the lower-left corner of a nominal 500 × 500 m tile, provided the LAS header bounds agree with that interpretation. This also makes neighbour discovery work for sparse sliver tiles whose actual point bounds cover only a tiny area.

---

3. Load the 10 m neighbouring-tile context

Language: Python with `laspy`, NumPy and memory mapping.

Core settings and commands:

```
None
NEIGHBOUR_HALO = 10.0
records = np.memmap(cache_path, dtype=HALO_CACHE_DTYPE, mode="r")
```

For each target file, the tool loads only points from adjacent files that lie within 10 m of the target boundary.

This means:

- Adjacent points influence seed selection, triangles, slopes and local planes.
- Only the target file's points are written to its output.
- Neighbour coordinates are never copied into the output.
- A neighbour file is decompressed/read once into a temporary cache and reused by all adjoining targets.

This prevents discontinuities caused by treating every LAS tile as an isolated island.

There is also a separate 10 m terrain-neighbour radius:

```
None
TERRAIN_NEIGHBOUR_RADIUS = 10.0
```

That radius is used for local plane fitting within the combined working cloud. The external 10 m halo and the local 10 m plane search are related but distinct controls.

---

4. Create internal processing windows

Language: Python and NumPy.

Core settings:

```
None
BLOCK_SIZE = 500.0
BLOCK_OVERLAP = 80.0
```

Large input files are divided into non-overlapping 500 m core windows. Each window is classified using an additional 80 m of internal context on all sides:

```
None
bxmin = core_xmin - BLOCK_OVERLAP
bxmax = core_xmax + BLOCK_OVERLAP
```

The 80 m area is classification context. Only the 500 m core result is retained.

Why 80 m?

- The initial seed grid is 60 m.
- A seed or TIN triangle close to a window boundary needs terrain on the other side.
- An overlap larger than 60 m allows the normal seed/TIN logic to operate across the boundary.
- Adjacent windows overlap in their calculations, but not in their output.

Therefore:

- Internal overlap: context between processing windows inside one file.
- Neighbouring-tile context: context obtained from separate LAS/LAS files.
- Ground envelope: a vertical/perpendicular tolerance around the final terrain surface.

These are three different concepts.

---

5. Select the source classes

Language: Python with NumPy Boolean masks.

Core command:

```
None
reset_mask = (
    (classification == 2)
    | (classification == 3)
    | (classification == 4)
    | (classification == 5)
    | (classification == 13)
)

classification[reset_mask] = 0
source_mask = classification == 0
```

Classes 2, 3, 4, 5 and 13 are returned to class 0 before classification. Existing class 0 points are also included.

Other existing classes, including class 7, 12, 14 and 15, are preserved and excluded from ground construction.

No last-return restriction is imposed. A legitimate terrain return can have any return number.

---

6. Construct the lower-envelope working points

Language: Python with NumPy.

Core function:

```
None
work_indices = lowest_per_grid(
    ground_candidates,
    x, y, z,
    spacing=1.0,
)
```

The command uses:

```
None
np.floor()
```

```
np.lexsort()
```

to divide the data into 1 m XY cells and choose the lowest eligible point in each cell.

This is the first meaning of “lower envelope”: a spatial collection of locally low candidate points that approximates the underside of the point cloud.

It is not yet the final ground surface. It can contain:

- True terrain.
- Ditch or valley points.
- Low vegetation.
- Occasional unclassified low errors.

Existing class-7 points do not participate.

The current script does contain a conservative gross-low guard. It does not run a general low-point filter. A candidate is temporarily excluded only when it is at least 1 m below a clean local plane and has insufficient neighbouring support at the same level. It is changed to class 7 only if the completed TIN later confirms the same isolated negative residual.

Core local-plane calls include:

```
None
tree = cKDTree(support_xy)
distances, neighbours = tree.query(...)
coefficients = np.linalg.solve(ATA, ATz)
```

The local plane has the form:

z=ax+by+cz=ax+by+cz=ax+by+c

The script fits it twice. After the first fit, a median absolute deviation test removes inconsistent returns, and a second robust plane is fitted.

Continuous valleys and slopes have neighbouring support and should therefore survive this check.

---

7. Select initial TIN seeds

Language: Python with NumPy.

Core command:

```
None
seed_indices = lowest_per_grid(
    work_indices,
    x, y, z,
    spacing=60.0,
)
```

The lowest work point in each 60 m cell becomes an initial seed.

The 60 m spacing corresponds to TerraScan's maximum-building-size concept: the cell should be approximately as large as the largest object the initial surface needs to pass underneath.

If a sparse tile produces fewer than four seeds, the script progressively halves the seed spacing.

This matches the approach described by Axelsson: start with a coarse set of low seed points, then progressively densify the TIN. The paper suggests seed-grid sizes of approximately 50–100 m depending on the largest structure. See Sections 3.1–3.3 of 111_xxxiii-part4.pdf.

---

8. Build the Delaunay TIN

Language: Python with SciPy/Qhull and NumPy.

Core command:

```
None
tin = Delaunay(
    model_xy,
    qhull_options="Qbb Qc Qz Q12",
)
```

A TIN is a Triangular Irregular Network: a terrain surface made from non-overlapping triangles whose vertices are selected LiDAR points.

Delaunay triangulation attempts to avoid unnecessarily narrow triangles by using the empty-circumcircle rule.

Four synthetic corner points are added just outside the processing bounds so the TIN covers the full window. Their elevations are extrapolated from nearby terrain using:

```
None
np.linalg.lstsq()
```

The synthetic corners are calculation support only. They are never written or classified as ground.

---

9. Calculate the progressive-TIN tests

Language: Python with NumPy vector mathematics.

Important commands:

```
None
normal = np.cross(p1 - p0, p2 - p0)
signed_distance = np.einsum("ij,ij->i", query - p0, normal)
angle = np.degrees(np.arctan2(abs(distance),
nearest_vertex_distance))
```

For every candidate, the tool finds the triangle underneath it:

```
None
simplex = tin.find_simplex(query_xy)
```

It then calculates:

- Perpendicular distance from the point to the triangle plane.
- Angle between the point's plane projection and the nearest triangle vertex.
- Triangle slope.
- Candidate slope from the triangle's lowest vertex.
- Longest triangle edge.
- Triangle edge ratio.
- Whether the triangle is geometrically degenerate.

The core acceptance test is effectively:

```
None
accepted = (
    signed_distance >= -0.15
    and perpendicular_distance <= pass_distance
    and point_angle <= adaptive_angle_limit
    and terrain_slope <= 88
    and candidate_slope <= 88
)
```

This implements the principal distance-to-facet and angle-to-node tests described in Section 3.4 of the paper.

---

10. Progressively lift and densify the TIN

Language: Python.

Core function:

```
None
model_indices = run_ptd_pass(...)
```

Pass 1 uses:

```
None
Angle:    6 degrees
Distance: 1.40 m
```

Pass 2 uses:

```
None
Angle:    4 degrees
Distance: 0.40 m
```

During every iteration:

1. Build the current TIN.
2. test all remaining work candidates;

3. select the best candidate in each triangle during Pass 1;
4. add the accepted vertices;
5. rebuild the TIN;
6. repeat until no candidate is accepted or 20 iterations are reached.

This follows the central progressive-TIN logic in Axelsson: one acceptable point per facet is added and the TIN is repeatedly reconstructed.

The angle is reduced inside smaller triangles:

```
None
ratio = np.clip(longest_edge / reduce_edge, 0.0, 1.0)
angle_limit = base_angle * ratio
```

Therefore, a large triangle can accept reasonable terrain variation, while a small triangle becomes increasingly strict. This is one of the principal adaptive behaviours.

---

11. Recover terrain the strict TIN missed

Language: Python with NumPy, KD-trees and robust local planes.

Three corrections are used uniformly across the project; there are no geographical correction masks.

Lower-envelope recovery

```
None
-2.0 <= preliminary_HAG < -0.15
```

Supported work minima below the coarse TIN can pull it down into ditches or depressions.

Anchored upper-surface recovery

Candidates between approximately 0.10 and 1.25 m above the current TIN may be accepted when:

- They are within 10 m of existing terrain.
- The local plane is clean.
- Several nearby points support the same residual.
- Their residual is consistent with that plane.

This helps fill genuine ground gaps without climbing into vegetation.

Convex-hill recovery

A coarse triangle can cut through a convex hill, placing its crest more than 1.4 m above the TIN. An ordinary distance test can never reach it.

The hill recovery therefore uses the robust local lower-surface plane rather than imposing a maximum HAG above the current TIN.

This was added specifically to recover continuous sloping terrain beneath trees.

---

12. Remove canopy climbs and TIN spikes

Language: Python with NumPy and `cKDTree`.

Core functions:

```
None
prune_model_above_lower_plane(...)
prune_upward_model_spikes(...)
```

A model vertex is removed only when it is:

- Above the local neighbour median.
- Above a robust lower-envelope plane.
- Not supported by several nearby points at the same residual elevation.

This distinction is important:

- A genuine ridge has repeated spatial support.
- A vegetation spike is normally isolated relative to the local terrain.
- A rough natural surface can have higher plane RMSE, causing the pruning test to relax or abstain.

The pruning runs for as many as three passes.

---

13. Select the smooth representative TIN

Language: Python with NumPy, KD-trees and robust planes.

Core function:

```
None
model_indices = select_smooth_model_representatives(...)
```

The progressive stage determines where terrain exists. This stage determines which points should actually become the clean interpolation surface.

The script:

1. fits robust local planes through the lower-envelope points;
2. evaluates original source returns against those planes;
3. chooses the return closest to the plane in each terrain cell;
4. keeps approximately 0.8 m representative spacing on flat ground;
5. keeps approximately 0.6 m spacing on slopes of 10 degrees or more;
6. retains structural fallback vertices where the plane cannot provide a reliable replacement.

This produces a one-surface-thick TIN rather than a thick band of class-2 returns.

The ground envelope

The second and most important meaning of “envelope” is the thin acceptance band around the final representative TIN.

It is used to increase class-2 density without putting every return within 0.5 m into class 2.

For every source return, the script calculates its signed perpendicular residual rrr from the local TIN triangle:

r=(P−P0)⋅n^r=(P-P_0)\cdot \hat{n}r=(P−P0)⋅n^

where:

- PPP is the LiDAR point;
- P0P_0P0 is a triangle vertex;
- n^\hat{n}n^ is the unit triangle normal;
- positive rrr is above the surface;
- negative rrr is below the surface.

The base envelope changes with slope:

Terrain	Above TIN	Below TIN

Flat	0.040 m	0.060 m
Steep, at 30° or above	0.080 m	0.100 m

The tolerance is then tightened for small triangles:

s=0.72+0.28clip(E5,0,1)s=0.72+0.28\operatorname{clip}\left(\frac{E}{5},0,1\right)s=0.72+0.28clip(5E,0,1)

where EEE is the triangle's longest edge.

The final condition is:

−Ls≤r≤Us-Ls \le r \le Us−Ls≤r≤Us

This envelope is:

- Perpendicular to the triangle, not simply vertical.
- Slightly slope-dependent.
- Slightly wider below the surface than above it.
- Tighter where the TIN is already detailed.
- Much thinner than the 0–0.5 m low-vegetation class.

For example, on flat terrain with a large triangle, acceptable ground is approximately 4 cm above to 6 cm below the TIN. A point 20 cm above the TIN is therefore low vegetation, not ground.

This is what creates the "blanket" of class 3 immediately above the smooth class-2 surface.

Envelope versus HAG

They are not the same measurement:

- Envelope residual: shortest perpendicular distance to the triangle, used to decide class-2 membership.
- HAG: vertical height above the interpolated terrain, used for vegetation classification.

On flat ground they are almost identical. On steep slopes, perpendicular distance is more geometrically appropriate for identifying returns belonging to the surface.

---

14. Adaptively densify class 2

Language: Python with SciPy Delaunay and NumPy.

Core function:

```
None
completed_ground = select_adaptive_ground_membership(...)
```

First, points must pass the thin envelope.

The candidates are pre-thinned only at 0.15 m to remove near-duplicates. The script then constructs a separate XY density TIN:

```
None
density_tin, density_xy = build_planar_tin(...)
```

For each iteration:

- Inspect current triangle edge lengths.
- Stop locally when the triangle is sufficiently small.
- Add one best-fitting point per triangle.
- Rebuild the density TIN.
- Repeat for up to six iterations.

Target longest-edge sizes are:

Terrain	Target edge
Flat	0.96 m
Steep	0.63 m

Consequently, slopes receive more ground vertices than flat areas. This is adaptive TIN density, not a fixed “one ground point every metre” rule.

The clean representative TIN remains the HAG/interpolation surface. Additional thin-envelope points receive class 2, but they do not make the reference surface spiky.

---

15. Interpolate HAG

Language: Python with SciPy and NumPy.

Core operations:

```
None
simplex = tin.find_simplex(query_xy)
transform = tin.transform[simplex]
ground_z = barycentric_interpolation(...)
hag = point_z - ground_z
```

Barycentric coordinates determine the point's position inside its triangle and interpolate the triangle elevation at that XY coordinate:

HAG=Zpoint−ZTINHAG=Z_{\text{point}}-Z_{\text{TIN}}HAG=Zpoint−ZTIN

For the very small number of points outside the numerical TIN boundary, the nearest genuine model vertex provides the fallback ground elevation.

HAG is written as a 32-bit floating-point extra dimension:

```
None
las.add_extra_dim(
    laspy.ExtraBytesParams(
        name="HAG",
        type=np.float32,
    )
)
```

---

16. Assign final classes

Language: Python with NumPy masks.

Final rules:

Condition	Class
Representative TIN or accepted thin-envelope return	2
Remaining point with HAG ≤ 0.50 m	3
0.50 < HAG < 1.00 m	4

1.00 ≤ HAG < 2.00 m	13
2.00 ≤ HAG ≤ 200.00 m	5
HAG > 200 m	7
Confirmed isolated gross-low error	7

Existing classes outside the reset set remain unchanged.

The final write is:

```
None
las.classification = final_classification
las["HAG"] = final_hag
las.write(output_path)
```

LAS input produces LAS output; LAZ input produces LAZ output.

---

17. Parallel processing and safety controls

Language: Python `concurrent.futures`.

Across files:

```
None
ProcessPoolExecutor(max_workers=file_workers)
```

Within one large file:

```
None
ThreadPoolExecutor(max_workers=window_workers)
```

Processes are used across files because every file has independent large arrays. Threads are used for windows because they can share the file-level NumPy arrays.

The scheduler uses:

```
None
Maximum logical CPU budget: 80%
Maximum simultaneous files: 6
Maximum simultaneous windows: 4
Parallel RAM budget: 55%
Minimum RAM left free: 4 GB
```

Six files is a ceiling, not a requirement. The number is reduced automatically when estimated RAM is insufficient.

Nested BLAS thread pools are limited to one thread to prevent each worker from secretly starting many additional threads.

Relationship to the Axelsson paper and TerraScan

The following parts directly reflect the published progressive-TIN method:

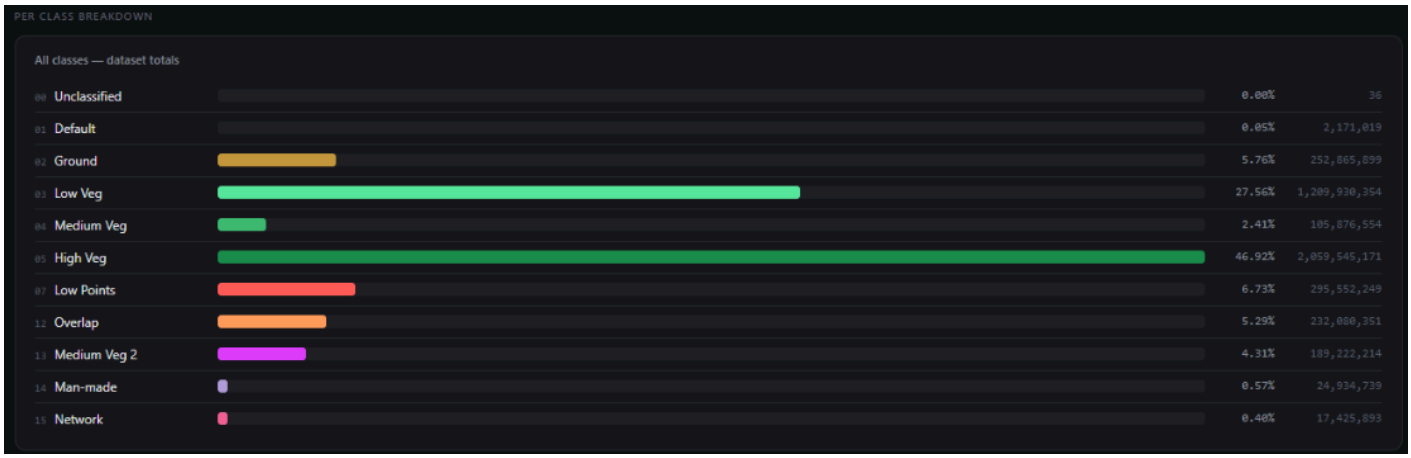
- Initial low points in a coarse grid.
- Initial TIN connected from below.
- Iterative TIN densification.
- Distance to the triangle plane.
- Angle relative to triangle nodes.
- One candidate per triangle during conservative densification.
- Rebuild the TIN after adding candidates.
- Stop when no more candidates are accepted.
- Optional density control through triangle-edge length.

These are described in Sections 3.1–3.4, printed pages 111–113, 111_xxxiii-part4.pdf.

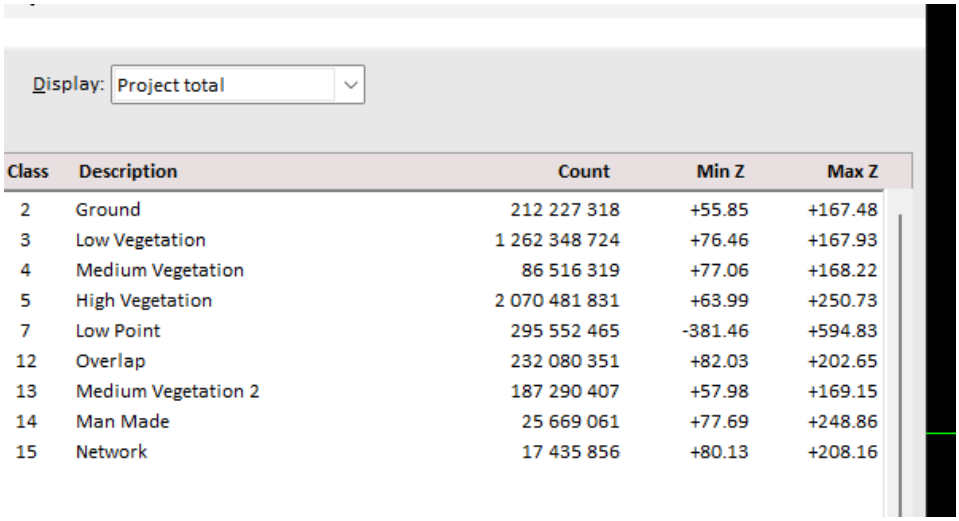
Implementation adds:

- Two calibrated PTD passes.
- 10 m neighbouring-tile context.
- 80 m internal window overlap.
- Robust 10 m local-plane fitting.
- Ditch and convex-hill recovery.
- Unsupported canopy/spike removal.
- Slope-adaptive smooth representatives.
- Thin adaptive ground membership.
- HAG vegetation classification.
- CPU/RAM-aware batch processing.
- Sparse/sliver-tile fallback and interrupted-run resume.

C98 HONI
Terra:



MBB:



Display: Project total

Class	Description	Count	Min Z	Max Z
2	Ground	212 227 318	+55.85	+167.48
3	Low Vegetation	1 262 348 724	+76.46	+167.93
4	Medium Vegetation	86 516 319	+77.06	+168.22
5	High Vegetation	2 070 481 831	+63.99	+250.73
7	Low Point	295 552 465	-381.46	+594.83
12	Overlap	232 080 351	+82.03	+202.65
13	Medium Vegetation 2	187 290 407	+57.98	+169.15
14	Man Made	25 669 061	+77.69	+248.86
15	Network	17 435 856	+80.13	+208.16